ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

Proyecto: Desarrollo de un prototipo web para la clasificación automática del estado de ocupación de estacionamientos integrando el modelo YOLOv11

Proyecto de Integración Curricular

1. Introducción

1.1 Propósito

El presente documento tiene como propósito definir, de manera formal y detallada, los requisitos funcionales y no funcionales del prototipo web para la clasificación automática del estado de ocupación en estacionamientos. Asimismo, establece los criterios de aceptación técnica basados en las métricas de precisión, sensibilidad y exactitud del modelo YOLOv11. Esta especificación constituye la guía fundamental para las fases de diseño, codificación y pruebas, asegurando la trazabilidad de los requisitos durante todo el ciclo de vida del desarrollo.

1.2 Alcance

a. **Identificación del producto.**
El producto de software objeto de esta especificación se denomina "Prototipo SmartPark". Se define como un prototipo web basado en visión artificial diseñado para automatizar el monitoreo de plazas de estacionamiento.

El sistema está compuesto por dos módulos principales:

- Módulo de Inferencia (Back-end): Desarrollado en Python con el framework Flask, encargado de procesar las imágenes mediante el modelo YOLOv11.
- Interfaz de Gestión Visual (Front-end): Una aplicación web desarrollada en Next.js que permite la interacción del usuario con los resultados del análisis.

b. **Funcionalidades.**
El sistema permitirá:

- La carga manual de imágenes en formatos estándar (JPG, PNG, WebP).
- Proporcionar herramientas para que el usuario delimite manualmente las coordenadas de los cajones de estacionamiento.
- Ejecutar algoritmos de preprocesamiento para optimizar la calidad de la imagen antes de la detección.
- Identificar y clasificar el estado de ocupación (Libre/Ocupado) de cada espacio delimitado.
- Presentar un reporte visual y estadístico del estado del estacionamiento.

El software no permitirá:

- Procesamiento de video en tiempo real.

- Gestionar el cobro o facturación de los estacionamientos.
- Reconocimiento de placas vehiculares (LPR).
- Controlar dispositivos físicos como barreras o sensores IoT.

c. Beneficios y metas.

Beneficios:

- Optimización del flujo de trabajo: Reduce el tiempo empleado en la configuración y delimitación de las Regiones de Interés (RoI) mediante herramientas de dibujo digital sobre la interfaz.
- Interpretabilidad de datos: Facilita la comprensión de los resultados de la inferencia mediante una representación gráfica en una interfaz web.

La meta principal es desarrollar una herramienta que integre la potencia de detección de YOLOv11 con una arquitectura web. Se busca demostrar que es posible la construcción de un sistema de clasificación de alta precisión a bajo costo que sirva como base para futuras implementaciones de movilidad urbana en el marco de las Smart Cities.

1.3 Acrónimos y abreviaturas

La siguiente tabla recoge, en orden alfabético, todos los términos, acrónimos y abreviaturas utilizados en el presente documento.

Término	Definición en el contexto del documento de especificación de requisitos
API	Interfaz de comunicación entre el front-end (Next.js) y el back-end (Flask) mediante solicitudes HTTP.
Back-end	Módulo de inferencia del prototipo SmartPark, desarrollado en Python con el framework Flask. Gestiona el preprocesamiento de imágenes y la ejecución del modelo YOLOv11
Bounding box	Rectángulo delimitador generado por YOLOv11 que envuelve el área de un vehículo detectado. Representado con coordenadas (x_min, y_min, x_max, y_max).
Cajón de estacionamiento	Espacio físico destinado al estacionamiento de un único vehículo. En el prototipo es representado por una ROI definida manualmente por el usuario sobre el lienzo (RF2).
Canvas / Lienzo	Área de dibujo interactiva de la interfaz web sobre la que el usuario renderiza la imagen cargada y traza los polígonos que representan cada cajón de estacionamiento (RF2).

Centroide	Punto central del bounding box de un vehículo, calculado como ((x_min + x_max) / 2, (y_min + y_max) / 2). El sistema determina la ocupación de una ROI cuando el centroide cae dentro de su polígono
CLAHE	Algoritmo de mejora de contraste adaptativo aplicado durante el preprocesamiento para mitigar efectos de sombras y variaciones lumínicas.
COCO	Dataset de referencia de visión computacional. En este prototipo se filtran únicamente las cuatro clases vehiculares de su taxonomía: car, truck, bus y motorcycle
Confianza (score)	Valor entre 0 y 1 asignado por YOLOv11 a cada detección, indicando la probabilidad de pertenencia a la clase predicha. Se aplica un umbral fijo de 0.45; detecciones por debajo de este valor son descartadas
Drag & Drop	Mecanismo de carga de archivos mediante el cual el usuario arrastra una imagen desde su sistema de archivos hasta la zona designada en la interfaz.
Exactitud	Proporción del total de predicciones correctas sobre el total de casos. Fórmula: (TP + TN) / (TP + TN + FP + FN).
FN	Cajón de estacionamiento ocupado que el modelo clasifica incorrectamente como libre.
FP	Cajón de estacionamiento libre que el modelo clasifica incorrectamente como ocupado.
Front-end	Módulo de interfaz de gestión visual del prototipo SmartPark, implementado con Next.js. Permite la carga de imágenes, la delimitación de ROIs y la visualización de resultados
Letterboxing	Técnica de reescalado que preserva la relación de aspecto original al redimensionar una imagen a 640×640 px, añadiendo relleno gris en los bordes sobrantes. Es el método de redimensionamiento estándar de la arquitectura YOLO.
Next.js	Framework de React utilizado para el desarrollo del front-end del prototipo. Provee la interfaz reactiva para carga de imágenes, delimitación de ROIs y visualización de resultados
Overlay	Capa semitransparente renderizada sobre la imagen original para indicar el estado de cada cajón: color verde para plaza libre y color rojo para plaza ocupada
Plaza libre	Estado de un cajón de estacionamiento en el que ningún centroide de bounding box detectado por YOLOv11 se encuentra dentro de la ROI correspondiente.
Plaza ocupada	Estado de un cajón de estacionamiento en el que el centroide de al menos un bounding box detectado por YOLOv11 se encuentra dentro de la ROI correspondiente.
Precisión	Proporción de plazas clasificadas como ocupadas que efectivamente lo estaban. Fórmula: TP / (TP + FP).
Preprocesamiento	Conjunto de transformaciones aplicadas a la imagen antes de la inferencia: reescalado a 640×640 px mediante Letterboxing, normalización de píxeles y aplicación del algoritmo CLAHE y filtro Gaussiano.
RF	Descripción de una capacidad observable del sistema desde la perspectiva del usuario.

RNF	Restricción de calidad medible que el sistema debe satisfacer. En este documento se definen tres, relativos a exactitud, sensibilidad y precisión del modelo.
ROI	Polígono trazado manualmente por el usuario sobre el lienzo para delimitar un cajón de estacionamiento individual. Es la unidad básica de análisis del sistema; la inferencia se aplica sobre cada ROI definida
Sensibilidad	Proporción de vehículos presentes en la imagen que son detectados correctamente por el modelo. Fórmula: TP / (TP + FN).
TN	Cajón de estacionamiento libre correctamente clasificado como libre por el modelo.
TP	Cajón de estacionamiento ocupado correctamente clasificado como ocupado por el modelo.
YOLOv11	Motor de inferencia del prototipo SmartPark. Última versión de la serie Ultralytics YOLO, también denominada YOLO11 en su documentación oficial. Detecta objetos en la imagen y genera bounding boxes con score de confianza para cada vehículo identificado

1.4 Referencias

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

2. Descripción General

2.1 Perspectiva del Producto

El producto es un prototipo independiente que interactúa con un servidor de inferencia basado en Python. La arquitectura sigue el patrón cliente-servidor, donde el cliente (Next.js) solicita el procesamiento de datos al servidor (Flask) que aloja el modelo de IA.

2.2 Funciones del Producto

Las funciones principales incluyen:

- Gestión de carga de archivos de imagen (WebP, JPG, PNG).
- Herramienta de delimitación manual de cajones de estacionamiento.
- Ejecución de algoritmos de preprocesamiento de imagen.
- Inferencia automática mediante el modelo YOLOv11.
- Visualización gráfica del estado de ocupación inmediato sobre la imagen.

2.3 Características de los Usuarios

El software está destinado a:

a. Investigadores en Visión Artificial y Aprendizaje Profundo:
   El software les permite:

- Realizar evaluaciones comparativas de las métricas de precisión, sensibilidad y exactitud de YOLOv11 frente a otros modelos.
- Utilizar el sistema como un entorno de experimentación controlado para validar la robustez del modelo ante variables externas, tales como cambios en las condiciones lumínicas, oclusiones parciales o variaciones en la resolución de las imágenes capturadas.

b. Desarrolladores:
Dirigido a ingenieros que buscan soluciones de bajo costo y alta escalabilidad para la gestión de infraestructuras urbanas. El software puede resultar útil para:

- Demostrar la viabilidad de la monitorización automatizada de plazas de estacionamiento sin depender de sensores físicos.
- Punto de partida para la integración de nuevas funcionalidades.

2.4 Restricciones

- El hardware debe tener capacidad suficiente para ejecutar inferencias de visión artificial (CPU/GPU).

3. Requisitos Específicos

3.1 Interfaces Externas

3.1.1 Interfaces de Usuario

3.1.2 Arquitectura del software

3.2 Actores

ID	Actor	Descripción	Requisitos asociados
AC1	Usuario	Persona que interactúa directamente con la interfaz web del prototipo SmartPark. Es responsable de cargar las imágenes de estacionamiento, delimitar manualmente las regiones de interés (ROIs) sobre el lienzo, iniciar el proceso de inferencia y consultar los resultados de ocupación.	RF1, RF2, RF5, RF6

AC2	sistema	Componente back-end del prototipo, implementado con el framework Flask sobre Python. Actúa como actor sistema: recibe las solicitudes HTTP enviadas por el front-end (Next.js), ejecuta el preprocesamiento de la imagen (RF3), lanza la inferencia mediante el modelo YOLOv11 (RF4) y retorna los resultados de ocupación al cliente de forma autónoma, sin intervención humana adicional.	RF3, RF4, RF6

3.3 Requisitos Funcionales

Id RF	Nombre del requisito	Descripción del requisito
RF1	Gestión de carga de imágenes	El sistema debe permitir al usuario la carga de imágenes de estacionamientos en formatos .webp, .jpg, .jpeg y .png, validando el tipo y tamaño del archivo.
RF2	Delimitación manual de regiones de interés	El sistema debe permitir al usuario trazar y gestionar manualmente los polígonos (regiones de interés) que representan cada plaza de estacionamiento, a través de una interfaz gráfica sobre el lienzo.
RF3	Preprocesamiento de imágenes	El sistema debe ejecutar automáticamente el reescalado, la normalización de píxeles y el ajuste de contraste de la imagen
RF4	Análisis de ocupación mediante YOLOv11	El sistema debe procesar la imagen mediante el modelo YOLOv11 para detectar vehículos y determinar el estado (libre/ocupado) de cada región de interés definida.
RF5	Visualización de resultados	El sistema debe renderizar sobre la imagen original los estados de ocupación mediante capas de color y etiquetas informativas.
RF6	Reporte de métricas de ocupación	El sistema debe calcular y mostrar un resumen estadístico detallando el total de plazas, el conteo por estado y el porcentaje de uso.

a. Criterios de aceptación

Por cada requisito funcional se establecen los criterios de aceptación.

Id RF	Criterios de aceptación
RF1	1. La interfaz debe incluir una zona de "Arrastrar y Soltar" (Drag & Drop) y un explorador de archivos.
	2. El sistema debe validar extensiones (.webp, .jpg, .jpeg, .png) y emitir un mensaje de error ante formatos no soportados.
	3. El sistema debe rechazar archivos mayores a 10MB

	4. Se debe mostrar una barra de progreso visual durante la subida.
	5. El usuario debe tener la opción de descartar la imagen y cargar una nueva.
	6. Tras una carga exitosa, el sistema debe habilitar un botón de navegación hacia el módulo de "Delimitación Manual de regiones de interés".
RF2	1. El sistema debe renderizar la imagen en un lienzo (canvas)
	2. El usuario debe poder dibujar, editar posición y eliminar polígonos (regiones de interés) sobre el lienzo.
	3. Las polígonos trazadas deben mostrarse con un borde de color neutro antes del procesamiento.
	4. El sistema debe permitir borrar un polígono mal trazado o resetear el lienzo por completo
	5. El botón "Ejecutar Inferencia" debe permanecer inhabilitado hasta que exista al menos un polígono definido.
	6. La interfaz debe contener un panel informativo que guíe al usuario en el proceso de delimitación manual, detallando las herramientas disponibles para el trazado de los espacios de estacionamiento.
RF3	1. El sistema debe redimensionar la imagen a 640x640 px mediante técnica de Letterboxing.
	2. Se debe aplicar el algoritmo CLAHE para mitigar efectos de sombras y un filtro Gaussiano para reducción de ruido digital.
RF4	1. El sistema debe ejecutar la inferencia con un umbral de confianza fijo de 0.45.
	2. Se deben filtrar únicamente las clases de la taxonomía de COCO: car, truck, bus, motorcycle.
	3. La lógica de ocupación se validará mediante la presencia del centroide del vehículo dentro de la ROI trazada en el RF2.
RF5	1. Las plazas deben teñirse con un overlay semitransparente: Verde (Disponible) y Rojo (Ocupado).
	2. Cada plaza debe mostrar un rótulo de texto con su estado y el porcentaje de confianza de la detección.
RF6	1. El panel de resumen debe mostrar: Total de plazas, plazas ocupadas y plazas disponibles.
	2. Se debe mostrar el porcentaje de ocupación con una precisión de un decimal (ej. 85.3%).

3.4 Requisitos de Rendimiento (requisitos no funcionales)

Id RNF	Categoría	Descripción del requisito	Criterio de aceptación
RNF1	Exactitud	El modelo debe alcanzar una exactitud (Accuracy) superior al 95% en la clasificación del estado de ocupación de plazas.	Medido sobre el dataset de prueba definido en el plan de pruebas, usando la fórmula (TP+TN)/total.
RNF2	Sensibilidad	El modelo debe mantener una sensibilidad (Recall) superior al 90% en la detección de vehículos presentes.	Medido como TP/(TP+FN) sobre el mismo dataset.
RNF3	Precisión	El modelo debe alcanzar una precisión (Precision) mínima del 92% en la detección de plazas ocupadas.	Medido como TP/(TP+FP) sobre el mismo dataset